

JavaScript-14

Installation av React

30 september 2026

JavaScript – utan tjafs
Författare: Acke Strömberg



För en kanin

Installation av React JS

När man bygger riktiga React-appar skapar man inte bara en lös main.js-fil. Man använder ett modernt byggverktyg som paketerar appen, hanterar externa bibliotek och gör sidan blixtsnabb. Branschstandard för detta heter Vite.

I det här kapitlet ska vi installera React med hjälp av Vite, titta på projektfilerna du får och starta upp din allra första lokala utvecklingsserver.

Installation steg-för-steg

Innan du börjar måste du ha Node.js installerat på din dator.

Öppna din terminal eller kommandotolk, navigera till mappen där du vill spara ditt projekt och kör följande kommando:

```
npm create vite@latest mitt-react-projekt -- --template react
```

Bild 1 Startkommandot för att skapa ett nytt React JS-projekt med Vite.

Vad händer nu?

1. Terminalen kommer att fråga om du vill installera paketet create-vite. Tryck Enter (Ja).
2. Du får välja språk. Använd piltangenterna, välj JavaScript (inte TypeScript!) och tryck Enter.
3. Navigera in i din nya projektmapp och installera alla grundfiler genom att köra dessa kommandon:

```
cd mitt-react-projekt  
npm install
```

Bild 2 Navigera in i rätt folder och installera alla npm-paket.

Nu är allt installerat! För att starta upp appen på din dator skriver du:

```
npm run dev
```

Bild 3 Starta upp utvecklingsservern.

Terminalen kommer att ge dig en lokal länk (oftast <http://localhost:5173/>). Öppna den i din webbläsare för att se Vites färdiga start-app.

Förklaring av projektstrukturen

När du öppnar projektmappen i VS Code kommer du att se ett gäng filer. Det kan se överväldigande ut, men det är faktiskt ganska enkelt. Här är de viktigaste filerna du behöver ha koll på.

```
mitt-react-projekt/
|-- node_modules/      # Innehåller alla externa bibliotek (rör ej!)
|-- public/            # För bilder och filer som inte ska ändras av kod
|-- src/               # HÄR SKA VI VARA! All vår kod ligger här.
|   |-- assets/        # Bilder och logotyper som används i koden
|   |-- App.css        # CSS-styling för vår huvudkomponent
|   |-- App.jsx        # Huvudkomponenten (vår startpunkt för HTML/JSX)
|   |-- index.css      # Global CSS-styling för hela webbplatsen
|   |-- main.jsx       # JavaScript-filen som kopplar React till HTML-sidan
|-- index.html         # Själva HTML-sidan som visas i webbläsaren
|-- package.json       # Alla inställningar och bibliotek i projektet
|-- vite.config.js     # Inställningsfilen för Vite (behöver sällan röras)
```

Bild 4 Filstruktur för projektet.

I vanliga JavaScript-projekt använder vi .js. Men i React använder vi .jsx. Det extra X:et står för XML och talar om för Vite och din kodredigerare att den här filen innehåller en blandning av JavaScript och HTML-strukturer (JSX).

Din första egna Counter-komponent

Låt oss rensa bort Vites startkod och bygga något eget!

Öppna filen `src/App.jsx`, radera allt som finns i den och klistra in följande rena, enkla exempel på en klickräknare:

```
import { useState } from 'react';
import './App.css'; // Vi kan importera CSS direkt i vår JavaScript!

function App() {
  // Vi skapar ett state (minne) som startar på 0
  const [antal, setAntal] = useState(0);

  return (
    <div className="raknare-box">
      <h1>Min första React-app</h1>
      <p>Du har klickat på knappen nedan {antal} gånger.</p>

      { /* När man klickar på knappen körs setAntal och uppdaterar värdet */ }
      <button onClick={() => setAntal(antal + 1)}>
        Plussa på (+)
      </button>
    </div>
  );
}

// Vi måste exportera komponenten så att main.jsx kan hitta och starta den
export default App;
```

Bild 5 Ett komplett exempel på hur en komponent styrs av sitt eget lokala minne (state).

Spara filen och titta i webbläsaren. Sidan uppdateras automatiskt (Hot Module Replacement) och din klickräknare är live!
